HACK APERTUS 2026 - TRACK 2B

CiteOps

Evidence-gated incident briefs with Apertus. A sovereign gateway that lets a language model analyze operational events without letting generated text authorize its own evidence.

Author	Siddharth Srivastava
Repository	github.com/sidsri14/citeops
Model	swiss-ai/apertus-v1.5-8b
License	Apache-2.0 software; CC-BY-4.0 report
Evidence date	12 October 2026

1. Summary

CiteOps transforms a bounded bundle of operational log events into a structured incident brief. Its central rule is simple: every generated hypothesis and action must cite event IDs from the submitted evidence set. Deterministic code rejects unsupported citations, missing citations, unexpected fields, and malformed output.

Credentials and email addresses are redacted locally before inference. The application has zero runtime npm dependencies, runs as a non-root read-only container, and supports either a hosted demonstration endpoint or a private OpenAI-compatible Apertus server.

Measured result: A live Public AI run passed the strict schema-and-citation gate on all 5 synthetic evaluation cases. Median end-to-end latency was 16.6 seconds. This is contract-compliance evidence, not a claim that every causal interpretation is correct.

2. Problem and design

Operational logs are valuable context, but they are also hostile input. They may contain secrets, personal information, attacker-controlled strings, or too little evidence to justify a confident diagnosis. Generic completions can invent causes or actions with no traceable support.

1. Accept exactly `request_id`, `service`, and 1-50 events.
2. Validate every field and cap total message text at 20,000 characters.
3. Replace common credentials, bearer tokens, API keys, and emails locally.
4. Ask Apertus for a candidate six-field JSON brief, treating logs as untrusted data.
5. Validate exact keys, types, enums, lengths, and collection bounds.
6. Require every hypothesis and action to cite submitted event IDs; otherwise fail closed.

3. Architecture and sovereign deployment

```
browser -> validation -> local redaction -> Apertus -> schema gate -> citation gate -> operator
```

The target architecture is on-premise and can be air-gapped after the image and model weights are staged. `LLM_BASE_URL` points at an OpenAI-compatible Apertus service on the private network. The UI is packaged in the same container and loads no external scripts, fonts, analytics, or telemetry.

Transport rules

- External model endpoints require HTTPS.
- Plain HTTP is accepted only for loopback, private, link-local, and local/internal hosts.
- Model responses are capped at 1 MiB and inference has a fixed deadline.

Container controls

- Unprivileged runtime user and read-only root filesystem.
- Dropped Linux capabilities and `no-new-privileges`.
- Small `tmpfs`, health check, and zero third-party npm runtime packages.

The demonstration used Public AI's hosted endpoint. Redaction still occurred before transport, but that route is weaker for sovereignty and exists to make the prototype reviewable. It does not redefine the private deployment target.

4. Use of Apertus

Property	Implementation
Model	`swiss-ai/apertus-v1.5-8b`
Task	Incident summarization, bounded hypotheses, suggested actions, and explicit unknowns
Interface	OpenAI-compatible `/v1/chat/completions`
Decoding	Temperature 0
Authority boundary	The model proposes content; application code owns privacy, bounds, schema, and citation integrity

The system prompt defines event messages as untrusted data and requires exact JSON keys, bounded severity/confidence/risk enums, and copied evidence IDs. Prompt compliance alone is never trusted: every constraint is checked again in code.

> **Fixture boundary:** Fixture mode creates one deterministic UI preview, visibly labels it `fixture-preview`, sets `apertus_used: false`, and is disabled by the evaluation runner. No fixture output is reported as model performance.

5. Data and evaluation

Five synthetic cases were authored for this project: dependency timeout, database pool exhaustion, secret-bearing logs, prompt injection inside a log, and insufficient evidence. They contain no production data. CiteOps does not train or fine-tune the model.

Two evaluation layers

1. **Deterministic acceptance:** tests cover exact schemas, unknown and missing citations, duplicate IDs, redaction, prompt injection, request singleflight/conflicts, HTTPS rules, response and body bounds, Host/Origin validation, and fixture labeling.
2. **Apertus generation:** `npm run evaluate` calls the configured model for all five cases and records latency plus the share of raw completions that pass schema and evidence validation. It refuses fixture mode and refuses silent overwrite.

Setup	Metric	Result
Node 24.19.0 local	Node test runner cases	20 / 20 passing
npm audit	Known package vulnerabilities	0
GitHub Actions	Docker build, health, cited API smoke	Passed, run 38239291460
Apertus v1.5 8B	Schema-and-citation pass rate	5 / 5 (100%)

15.9s	16.6s	17.3s	19.4s
minimum	median	mean	maximum

The live run used Public AI on 12 October 2026 with temperature 0. Captured artifacts are in `data/model_responses.json` and `data/evaluation_results.json`. The API credential, raw synthetic email, raw password, and injected instruction are absent from those files.

Interpretation: 100% means every raw completion obeyed the exact output contract and cited only submitted event IDs. It is not a semantic accuracy benchmark and does not prove that a cited cause is true.

Per-case latency includes network transit and hosted inference. The five-case sample is intentionally small and synthetic; it demonstrates the evaluation path rather than production throughput.

6. Security and failure semantics

- Invalid model output fails closed; CiteOps does not silently repair unsupported claims.
- Secrets and email addresses are redacted before model transport.
- Log-borne instructions remain JSON event data and cannot change the system contract.
- Unknown evidence IDs, uncited hypotheses/actions, and extra output fields return HTTP 502.
- Request IDs are content-fingerprinted. Identical concurrent calls share one promise; conflicting reuse returns HTTP 409.
- Transient failures are removed from the idempotency map so they do not poison a request permanently.
- Browser POSTs require a same-host Origin when supplied; Host values must be allowed or local/private.

7. Limitations

Pattern-based redaction cannot recognize every organization-specific secret. Production deployment needs custom detectors and an existing identity-aware access layer. A valid citation proves only that referenced evidence exists, not that the model interpreted it correctly.

In the sparse evaluation case, Apertus proposed possible backend and logging causes even though the only event said "unspecified internal error." The output also preserved that uncertainty. This is why CiteOps is decision support, not an autonomous remediation agent.

Prototype omissions

- No RBAC or durable incident history.
- No SIEM connector or streaming inference.
- No command execution, tool calling, URL fetching, or remediation side effects.

Scale boundary

- Idempotency state is bounded and process-local.
- Multiple replicas need a shared store.
- Hosted validation does not prove the operator's private serving configuration.

Docker was unavailable on the development workstation, so the container path was independently built and smoke-tested in GitHub Actions at commit `54bf286`. The hosted inference path was validated separately through Public AI.

8. Reproducibility and next steps

Judges set `LLM_NAME`, `LLM_BASE_URL`, and `LLM_API_KEY`, then run `make run` from the repository root. The compose service exposes port 8080.

Command	Purpose
`npm test`	Run deterministic unit and HTTP integration tests
`npm run check`	Syntax-check all JavaScript source files
`npm audit`	Check the npm dependency graph
`npm run evaluate`	Run real non-fixture Apertus evaluation with exclusive result-file creation

The evaluation command can load an ignored local `.env.local`; credentials are never committed. Captured results record the model, timestamp, endpoint origin, and pass count without exposing the key.

Most useful next steps

- Organization-specific redaction policies and authenticated access.
- Durable audit history and a shared idempotency store.
- Human feedback labels and calibrated semantic evaluation.
- Read-only SIEM import connectors that preserve the no-remediation boundary.

References

1. Swiss AI Initiative, Apertus v1.5 model family: huggingface.co/swiss-ai/Apertus-v1.5-8B
2. Hack Apertus, Track 2B project requirements: hackapertus.notion.site/track-2-apertus-adoption
3. Public AI OpenAI-compatible API documentation: platform.publicai.co/docs
4. CiteOps source and captured evaluation artifacts: github.com/sidsri14/citeops

License

Software: Apache License 2.0. Report: Creative Commons Attribution 4.0 (CC-BY-4.0).

Submission statement: CiteOps uses Apertus for bounded semantic analysis while deterministic code retains control over privacy, schema, evidence integrity, and all operational side effects.